

DevPulse

Version-Safe Migration Intelligence

AI Product Management Case Study

How a hard-gate retrieval architecture makes wrong-version rate
mathematically 0.0 — not approximately 0.0

0.0

Wrong-Version Rate

0.966

Conflict F1

0.97

Recall@5

2,479

Backtest Queries

github.com/SidharthKriplani/devpulse_platform

Sidharth Kriplani · 2026

Production-simulated · RAG + Agentic · Version-Safe Retrieval

Executive Summary

DevPulse is a production-simulated, version-safe documentation retrieval and agentic dependency migration platform. It addresses a specific, consequential failure mode in LLM-assisted software development: the retrieval layer returning documentation from the wrong API version, causing developers to apply confidently wrong guidance. This case study examines the product decisions behind DevPulse — what was built, what was explicitly not built, and why each boundary was drawn with precision.

WRONG-VERSION RATE	CONFLICT F1	HYBRID RECALL@5	BACKTEST QUERIES
0.0	0.966	0.97	2,479
Deterministic guarantee	4-class detection	BM25 + dense fusion	37-day simulation

SECTION 1

The Problem

The User

Software engineering teams migrating between library or API versions. These are not novice developers — they are experienced engineers making deliberate decisions to reduce technical debt. They reach for AI-assisted tooling because the migration surface is large: changelogs span hundreds of entries, breaking changes are scattered across documentation, and the cognitive load of holding two API versions simultaneously is substantial.

The Failure Mode

LLM-based migration assistants hallucinate deprecated APIs. The root cause is not the LLM — it is the retrieval layer. The model does not know which version the team is on. A vanilla RAG system asked *'how do I migrate from requests v2 to v3?'* will retrieve chunks from multiple versions without filtering. The retrieval system returns by semantic similarity, and both v2 and v3 documentation are semantically close to the query.

The Scenario: A developer asks the assistant about migrating authentication patterns from requests v2 to v3. The retrieval layer returns three chunks: two from v3 (correct) and one from v2 (deprecated pattern). The LLM synthesizes across all three without awareness of the version mismatch. The output reads confidently correct. The developer applies it. Tests fail. The root cause is invisible — the failure appears as a test regression, not as a version mismatch.

The consequence is a silent regression introduced during a migration that was supposed to *reduce* technical debt. The engineering team has less confidence in the AI tooling, the migration stalls, and the wrong pattern may propagate across multiple callsites before anyone identifies the source.

Why This Is a Product Problem, Not an Engineering Problem

The instinct is to frame this as a retrieval quality problem — improve ranking, tune reranking weights, add more training data. But that framing misses the structural issue. A better-ranked retrieval system still returns wrong-version content: it just ranks it slightly lower. The developer cannot see the version provenance of what the LLM synthesized. The confidence of the output is indistinguishable whether the retrieval was version-safe or not.

The product question is: **can we make the wrong-version failure mode structurally impossible, rather than statistically unlikely?** That is the question DevPulse answers.

The Version-Crossing Failure

Stage	Vanilla RAG	DevPulse
-------	-------------	----------

Query received	Parsed for semantic embedding	Version tag extracted first
Retrieval	Top-k by similarity — all versions eligible	Hard gate: non-target versions excluded before scoring
LLM input	Mixed-version chunks passed to LLM	Version-clean chunks only, after conflict check
Output	Confident, potentially wrong-version guidance	SAFE / RISKY / BLOCKED with version citations
Wrong-version rate	Statistically reduced, never 0.0	0.0 — deterministic

SECTION 2

The Product Decisions

The Core Decision: Hard Gate vs. Soft Reranking

Every product decision in DevPulse flows from a single architectural choice: version filtering operates as a **hard gate** before retrieval, not as a soft reranking signal after retrieval. This is the load-bearing decision.

Hard Gate: Chunks tagged to incompatible versions are excluded from the candidate pool before BM25 or dense scoring runs. No wrong-version chunk can reach scoring, reranking, or the LLM — regardless of its semantic similarity score. Wrong-version rate = 0.0, deterministically. **Soft Reranking:** All chunks enter the candidate pool. Wrong-version chunks receive a penalty in the reranking score. They rank lower — but they can still rank in the top-k if their semantic similarity is high enough. Wrong-version rate $\approx$ 0.0, probabilistically.

The distinction matters for trust, not just accuracy. A system that is probabilistically version-safe requires developers to assess: *how confident am I that the reranker penalized the wrong-version content sufficiently on this specific query?* That question has no answer a developer can act on. A system with a hard gate removes the question entirely. The reliability guarantee is structural, not statistical.

What Was Built

- **Version filter as a hard gate.** Every document chunk carries a `version_tag`. When a query specifies a target version, the retrieval layer applies a hard filter: non-matching chunks are excluded before scoring. This makes wrong-version rate mathematically 0.0 across all 2,479 backtest queries.
- **Conflict detector across 4 types.** Before any LLM synthesis, the conflict detector classifies retrieved chunks across: Stale (chunk version older than target), Contradictory (two same-version chunks make incompatible claims), Deprecated (chunk explicitly marks API as deprecated), and Cross-source (changelog and migration guide disagree). Macro F1: 0.966.
- **LLM-Last principle.** The LLM is the final component in the pipeline, not the first. Synthesis is gated behind: version filter active, conflict detector returned CLEAR, at least one grounded chunk retrieved, staleness checks passed. If any gate fails, the system returns a deterministic RISKY or BLOCKED verdict without calling the LLM. The LLM cannot hallucinate — it only synthesizes over pre-validated, version-pinned evidence.
- **SAFE / RISKY / BLOCKED report with citations.** Every migration recommendation includes a structured verdict, the conflict evidence that determined it, and citations assembled from retrieved chunks — not generated by the LLM.

- **Repo-aware callsite scanner.** The system scans a local repository against the migration target, mapping dependency usage across source files and flagging risky callsites. Conservative by default: outputs carry `DO_NOT_APPLY_WITHOUT_REVIEW`.
- **Patch and PR simulation.** DevPulse generates reviewer-safe migration artifacts: structured patch proposals, reviewable diffs, risk assessments, and pre-patch test state captures. The output is a PR that a human engineer reviews, not a mutation the system applies autonomously.

What Was Explicitly Not Built

The boundaries of DevPulse are as deliberate as its features. Three categories were explicitly excluded:

Not Built	Why Not
Live npm / PyPI registry integration	Would couple system reliability to external uptime. When the registry is slow or unavailable, the migration pipeline stalls. The version intelligence needed for migration decisions does not require live registry calls — it requires version-tagged documentation, which is static.
Autonomous code mutation	The system proposes; engineers decide. Autonomous mutation removes the engineer from a decision with non-trivial production risk. Patch and PR simulation preserves the engineer's agency while reducing the work of constructing the migration.
Real CI execution	DevPulse captures pre-patch test state and simulates post-patch expectations. Running actual CI would require environment provisioning, secrets, and infrastructure that belong outside the migration intelligence layer.

The Failure Mode Addressed

The Core Question DevPulse Answers

"Can I trust this migration recommendation?"

Every design decision in DevPulse flows toward a single answerable question. Not *'is this recommendation likely to be correct?'* — that is a probabilistic framing that leaves the trust burden on the developer. DevPulse's answer is structural: the recommendation is trustworthy because wrong-version content was excluded before it was generated, conflicts were detected before synthesis ran, and the verdict is derived from version-pinned evidence, not LLM inference.

The Version-Crossing Failure Mode

The version-crossing failure mode is distinct from ordinary RAG quality problems. It does not require poor retrieval — it occurs even in high-quality retrieval systems. Here is why: semantic similarity does not respect version boundaries.

Documentation for `authenticate()` in v2 and v3 of the same library will be semantically very similar — same function name, same domain vocabulary, similar structure. A dense retrieval system scores both highly for a query about `authenticate()`. The v2 chunk may rank above some v3 chunks. The LLM receives both. The synthesis is confident and internally coherent — but may reflect v2 behavior.

The version-crossing failure is not a retrieval quality problem. It is a retrieval architecture problem. Improving retrieval quality — better embeddings, better reranking — reduces the probability of version crossing but cannot eliminate it structurally. A hard gate eliminates it structurally.

The Four Conflict Types

Even within a version-safe retrieval pool, chunks can conflict. The conflict detector classifies four distinct types, each representing a different trust risk:

Conflict Type	Description	Trust Implication
Stale	Chunk version older than target; may contain superseded guidance	RISKY — guidance may not apply to target version
Contradictory	Two chunks from same version make incompatible claims	BLOCKED — synthesis suppressed; evidence returned only

Deprecated	Chunk explicitly marks an API or pattern as deprecated	RISKY — developer must explicitly acknowledge deprecation
Cross-source	Changelog and migration guide disagree on same version behavior	BLOCKED — documentation source conflict; unsafe to synthesize

The BLOCKED Verdict as a Product Feature

Most AI-assisted tools surface a recommendation regardless of evidence quality — because a non-answer feels like a product failure. DevPulse inverts this. A BLOCKED verdict is not a failure state: it is the system correctly identifying that the evidence is insufficient to make a safe recommendation. Suppressing synthesis when evidence is contradictory is harder to ship than returning a hedged answer — but it is the decision that makes the system trustworthy in the cases that matter most.

SECTION 4

What Success Looks Like

Product Metrics

Success for DevPulse is defined at the product layer, not the engineering layer. The metrics that matter are not model accuracy scores — they are the outcomes that change developer behavior and migration quality.

Wrong-Version Answer Rate	0.0
The version filter hard gate makes this mathematically 0.0 across all queries — not approximately 0.0 on a benchmark. This is the primary trust metric. If this is non-zero, the system's core promise is broken.	<i>Deterministic guarantee</i>
Engineer Time Saved Per Migration Decision	Target metric
A developer using DevPulse should reach a confident migration decision — with version-cited evidence and a clear verdict — faster than manually cross-referencing changelogs, migration guides, and Stack Overflow. The target is to reduce decision time from hours to minutes for individual API migrations.	<i>Time to confident decision</i>
Migration-Introduced Regressions	Reduction target
The version-crossing failure mode causes regressions that are hard to attribute. Success is a measurable reduction in the rate of test failures introduced during AI-assisted migrations compared to migrations conducted without version-safe retrieval. This is the outcome metric that justifies the product.	<i>Post-migration defect rate</i>
PR Review Time for AI-Generated Patches	Efficiency metric

DevPulse generates reviewer-safe migration patches with structured risk assessments and pre-patch test state. Success is PR review time shorter than for manually-authored migration PRs — because the evidence package is already assembled and the risk surface is documented.

Reviewer time to merge or reject

The Backtest Baseline

A 37-day simulated backtest across 2,479 queries provides the baseline for the wrong-version rate guarantee and retrieval metrics. The backtest uses query templates generated across the full golden evaluation category set — covering all four conflict types, all three verdict states, and both simple and migration-complexity query modes. Hybrid Recall@5 (BM25 + dense fusion) reached 0.94 against the golden set; the reranker simulation lifted this to 0.97.

Agentic Goal Mode — The Migration Plan Output

Goal Mode extends DevPulse from single-query decisions to full dependency migration plans. Given a `package.json` or `requirements.txt`, the system parses all dependencies, classifies semver deltas (patch / minor / major), plans an ordered migration queue with risk ordering, dispatches Query Mode queries per dependency, applies bounded recovery logic for blocked tasks, and produces a staged migration recommendation with aggregate verdict.

Success for Goal Mode is measured by whether the staged migration plan reduces the number of migration decisions an engineer must make manually — while preserving the engineer's authority over each change.

SECTION 5

One Honest Gap

What DevPulse Does Not Detect

DevPulse detects version tag mismatches. It does not detect semantic equivalence across versions.

Two chunks from the same version that contradict each other in subtle semantic ways may both pass the version filter. The Contradictory conflict type catches explicit incompatibilities — two chunks that make directly opposing claims about the same API. But subtle semantic drift within a version — where two chunks make claims that are technically compatible in isolation but produce inconsistent developer behavior in combination — is not detected.

The gap in precise terms: DevPulse's version filter is a structural guarantee against version-crossing. It is not a semantic coherence guarantee within a version. Two chunks tagged v3.2.1 that give subtly inconsistent guidance about optional parameter behavior will both pass the filter and both enter the conflict detector — which will classify them as CLEAR if neither makes a directly incompatible claim.

Why This Gap Exists Deliberately

Closing this gap would require embedding-based semantic comparison across chunk pairs within the retrieved set. That comparison is probabilistic — it introduces a confidence threshold above which the system classifies two chunks as semantically contradictory. Choosing that threshold is a judgment call that reintroduces probabilistic error into a system explicitly designed to make deterministic guarantees.

The scope boundary is deliberate. DevPulse makes one strong deterministic guarantee (version-crossing rate = 0.0) rather than two weaker probabilistic guarantees (version-crossing $\approx$ 0.0 and intra-version semantic contradiction $\approx$ 0.0). A system that is deterministically correct on the primary failure mode and honest about its secondary limitations is more trustworthy than a system that claims broader coverage it cannot guarantee.

What a Follow-On Product Would Require

Semantic conflict detection within a version is a distinct product problem. It requires: a ground truth set of within-version semantic contradictions (currently absent from most documentation corpora), an embedding model fine-tuned on technical documentation contradiction rather than general semantic similarity, and a threshold calibration methodology that maintains acceptable false-positive rates without suppressing too many valid SAFE verdicts. These are solvable problems — but they are not the same problem DevPulse solves, and conflating them would weaken the guarantee DevPulse currently makes.

DevPulse — Version-Safe Migration Intelligence

github.com/SidharthKriplani/devpulse_platform

Production-simulated · RAG + Agentic Orchestration · Version-Safe Retrieval · 70 Evidence Artifacts